

Fully-Tested code generation from TLA⁺ specifications

Gabriela Moreira
gabrielamoreira05@gmail.com
Santa Catarina State University
Joinville, SC, Brazil

Cristiano Damiani
Vasconcellos
cristiano.vasconcellos@udesc.br
Santa Catarina State University
Joinville, SC, Brazil

Janine Kniess
janine.kniess@udesc.br
Santa Catarina State University
Joinville, SC, Brazil

ABSTRACT

Specifying software systems brings value by increasing confidence. However, engineers struggle to adopt this practice for its steep learning curve and lack of connection to production software. These problems can be addressed with simulation of specifications and generation of test cases. TLA⁺ (Temporal Logic of Actions⁺), a specification language used by big tech companies, is still mostly devoid of this tooling. This work proposes a combination of code and test generation from TLA⁺ files with the goal of making specification writers obtain value from their specifications as soon as possible.

KEYWORDS

Formal specification, Code generation, Test generation, Model-Based testing, TLA⁺, Elixir

ACM Reference Format:

Gabriela Moreira, Cristiano Damiani Vasconcellos, and Janine Kniess. 2022. Fully-Tested code generation from TLA⁺ specifications. In *VII Brazilian Symposium on Systematic and Automated Software Testing (SAST 2022)*, October 3–7, 2022, Uberlândia, Brazil. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3559744.3559747>

1 INTRODUCTION

Formally specifying software is a valuable practice to increase trustability on software, providing mechanisms to verify properties of behaviors at the software's design stage. The importance of this verification process becomes imperative when designing concurrent systems, where the number of reachable states and behaviors is not fit to the designer mind alone. Therefore, specification methods arise with focus on concurrent systems, such as Petri nets [20] and the pi calculus [18].

Following these works, Lamport proposes TLA⁺ in [12] based on the TLA (Temporal Logic of Actions) logic, which extends temporal logic terms allowing predicates over pairs of states, called actions. TLA⁺ provides a declarative and mathematical syntax to define actions that together will dictate whether a behavior - an infinite sequence of states - is accepted or not by the specification. Model checkers can be used to verify properties of a system modeled in TLA⁺, and there is currently two tools with different strategies available: TLC [14] and Apalache [10].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SAST 2022, October 3–7, 2022, Uberlândia, Brazil

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9753-7/22/10...\$15.00

<https://doi.org/10.1145/3559744.3559747>

Besides design validation, writing a formal specification can also bring benefits to existing running systems. It can help to find bugs and inconsistencies, as well as allowing for substantial optimizations by changing the specification and checking whether it still satisfies the set of properties. These benefits were reported by Amazon Web Services [19] after using TLA⁺ in 10 of its complex systems and, for each one of them, finding bugs or understanding about the implementation that led to optimizations.

Written formal specifications, however, lack any correspondence with the actual implementation besides the programmer's understanding. Some known tools to mitigate this problem are trace checking, code generation and test case generation. Code generation, where executable code is generated from the specified model, is popular due to the automation of manual work, and it has implementations for some existing formal specification languages such as Z, B-Method and ASM (Abstract State Machine). TLA⁺ however still lacks most of this tooling, with no published materials on this matter as of the writing of this paper¹. As for test case generation, there are two separate initiatives on model-based testing from TLA⁺ models: exploring TLC state space [6] and reproducing witness from Apalache's counterexamples or simulation [11].

The tool proposed in this work does both code generation and test case generation. It takes a specification written in TLA⁺ and generates Elixir [22] code, unit tests and integration tests, with the main goals of maintaining a cognitive correspondence between actions (from the specification) and functions (from the generated code), prioritizing readability and facilitating the deployment in distributed and concurrent scenarios. By combining code and test generation in the same tool, test harness becomes trivial and tests work out-of-the-box without the need to define any integration code with the implementation.

When a bug is found by model checking, it is often not trivial how to 1. Reproduce that bug in production software, and 2. Ensure the bug is fixed after an update to the implementation. This is specially hard when the person writing the specification is not the same as the one implementing the software. By providing plugable interfaces in the generated code and integration tests with capability of reproducing traces, the tool described in this work aims at reducing the gap between model and implementation with this problematic as focus.

This work presents a TLA⁺ basics overview at Section 2 with the main concepts required to understand the specification format that is used for generation. Section 3 introduces the proposed translation, explaining how specification concepts are represented in executable forms. Section 4 explains the two testing strategies

¹There is a code generation tool for PlusCal, which is a more imperative language that translates to TLA⁺ [26]. This tool cannot be used to generate code from TLA⁺.

derived from different model checker information. Section 5 explains an evolution to the presented strategies that make them more adept to decentralized systems, and Section 6 walks through how this evolved strategy can be applied to a real decentralized algorithm. Finally, related work is mentioned in Section 7 and a conclusion along with future work intents is presented in Section 8. The tool implementation along with specification and generated artifacts for the example and other examples are available publicly at <https://github.com/bugarela/tla-transmutation>.

2 TLA⁺ OVERVIEW

TLA⁺ is a specification language combining the TLA (Temporal Logic of Actions) logic and the Zermelo-Fraenkel set theory (ZFC) [17]. TLA was also proposed by Lamport [13] and is based on state transitions, where a set of allowed transitions is modeled as one or more actions. An action is a predicate over a pair of states, which is called a step. If an action is true for a step from $s \rightarrow s'$, then a transition from $s \rightarrow s'$ is part of the transition set of the model.

Modeling a system with TLA means specifying a set of possible behaviors - sequences of states - in that system. This is done by specifying a temporal formula that is true for all the steps in those behaviors, and false otherwise. A system behavior can be checked against other temporal formulas, in such a way that the formula is true for the behavior if it is true for every step of the behavior.

In a TLA⁺ spec, a system can be defined with two formulas: a predicate over the initial state, defining a set of allowed initial states; and an action (often called *Next*) over the system steps, defining a set of allowed transitions. An action can be interpreted as a conjunction of predicates over a current state and predicates over the next state. In this paper, predicates over the current state will be called an action's condition - meaning they represent a condition for the transition to happen from that state.

As an example of basic TLA⁺ usage, presented in Figure 1, consider a traffic semaphore with three states (red, yellow, green) and where the set of behaviors contains all behaviors that allow red to turn into green, green to turn into yellow, and yellow to turn into red, beginning in any of those states. This could be written as the *TrafficSemaphore* TLA⁺ spec.

MODULE *TrafficSemaphore*

VARIABLE *color*

$TurnRed \triangleq color = \text{"yellow"} \wedge color' = \text{"red"}$
 $TurnYellow \triangleq color = \text{"green"} \wedge color' = \text{"yellow"}$
 $TurnGreen \triangleq color = \text{"red"} \wedge color' = \text{"green"}$

$Init \triangleq color \in \{\text{"red"}, \text{"yellow"}, \text{"green"}\}$

$Next \triangleq TurnRed \vee TurnYellow \vee TurnGreen$

Figure 1: TLA⁺ Specification for the traffic semaphore

In the *TrafficSemaphore* spec, *color* is the value for the variable color in the first state of a step, while *color'* is the value for the second state. The *Next* definition lists three possible actions, and each one of them has a condition (predicate over the current color), and a transition (which color to turn to). Although the TLA⁺ IDE

allows the definition of a spec with the identification of an *Init* and a *Next* definitions, the same spec could also be defined with the temporal formula:

$$Spec \triangleq Init \wedge \Box Next_{color}$$

TLA⁺ syntax borrows elements from first-order logic and set theory. The logical operators $\wedge$ and $\vee$ can be used both prefixed and infix such that:

$$\begin{array}{lcl} \wedge e_1 & & \vee e_1 \\ \vdots & \equiv e_1 \wedge \dots \wedge e_n & \text{and} \quad \vdots \equiv e_1 \vee \dots \vee e_n \\ \wedge e_n & & \vee e_n \end{array}$$

2.1 Stuttering Steps

The *Next* definition in the presented temporal formula is written as *Next_{color}* as a way to specify that *Spec* allows stuttering steps over the variable color. A stuttering step represents self-transitions, such that an action stating $color = color'$ is always allowed. Stuttering steps are important to represent loops and crash states. Considering the traffic semaphore example, allowing stuttering steps over *color* means the semaphore could get stuck forever in any of the possible colors.

3 CODE GENERATION

From a TLA⁺ spec modeling a set of behaviors, its generated code is an executable code that is capable of executing the same set of behaviors. It can be useful for prototyping from a spec and validating the system's integration in the real world. The generation being proposed yields an Elixir module where each action is translated to two functions, in such a pattern that aims to preserve the spec structure and make it easier for the spec's designer to understand and manipulate the generated code.

Elixir is a functional programming language heavily based on Erlang [2] process communication and Ruby [4] syntax. The motivation for generating Elixir code instead of any other language is based on several reasons:

- (1) Reliable concurrency and inter-process communication from Erlang's virtual machine (BEAM) usage, which is essential to support concurrent and distributed systems that are a TLA⁺ focus.
- (2) The function paradigm is declarative over operational, same as the TLA⁺ syntax. This helps to keep a correspondence between definitions in each language.
- (3) Elixir is focused on legibility and maintainability, borrowing a lot from Ruby's syntax, which is desirable to provide an artifact that can actually be adapted and upgraded, making it more suitable to run in a real life environment.
- (4) BEAM provides platform transparency, making the generated code suitable for a bigger number of environments.
- (5) Elixir is open source under Apache 2.0 license, which is crucial to further ensure a correspondence between code and spec on all levels.

3.1 Translation

The process of transforming a specification into executable code is a translation, which can be expressed with translation rules that

map TLA⁺ specifications to Elixir code. A TLA⁺ specification is composed of some metadata such as module name, variables, constants and comments, along with a list of definitions. The outmost translation process extracts this metadata to a context Γ and use it to translate each definition to functions. A definition is composed of an identifier, zero or more parameters, and a body. If the body does not define any transitions, it is translated to a simple pure function. Otherwise, it is translated to two functions: one representing the action's condition, returning a boolean value if and only if the action is enabled in the given state; and one representing the action's transition, taking the values for all variables in the current state and returning their values on the next state. Values for state variables are defined in a key-value structure, a Hash in Elixir referred as *variables*, where keys are variable names and values are their value in a state.

An action can be defined by many actions combined with ($\wedge$) or ($\vee$) operators, which for simple predicates as conditions can be directly translated to the equivalent logic operator in Elixir. Aggregating transitions by conjunction ($\wedge$) results in their states being merged as if the aggregated transitions were composed. Aggregating transitions by disjunction ($\vee$) is not trivial, since it can represent non-determinism - possibly more than one transition can be executed. This is solved by delegating this non-determinism to a `decide()` function that interacts with an Oracle component, which is further explained at Section 3.4.

A set of detailed translation rules for code generation is defined at <http://bit.ly/tla-code-generation>.

The translation for the *TurnRed* definition from Figure 1 results in the following two functions:

```
def turn_red_condition(variables) do
  variables[:color] == "yellow"
end

def turn_red(variables) do
  %{ color: "red" }
end
```

3.2 Initial state

Just like the TLA⁺ tools, the translator needs to be informed about the initial state and next state definitions. The initial state definition has statements over a single state, which qualify a state to be a valid initial state of the program. An executable code, however, should have a unique initial state to begin from. Therefore, the translation requires an unambiguous initial state definition, meaning it should define exactly one possible state.

The initial state definition is translated to an Elixir hash mapping all variable names to their values - the initial value of *variables*. This is the first state of the executable program, and all other states will be represented the same way.

3.3 Next state

The next state action is the main definition of the system's states transitions. It is an action like any other, but it should not have any conditions. Upon informing the translation tool that an action is the next state action, its name will be translated as `next()`. It receives

the state, represented as a hash as its parameter, and returns a list of possible next states.

The generated `main()` function is always the same for every specification, and its job is to invoke `next()` repeatedly, requesting choices from the oracle whenever `next()` returns more than one state and notifying the oracle of every transaction it performs.

3.4 Oracle

A useful specification does not describe a single behavior, which could be easily verified by hand. Having multiple behaviors be converted to an executable code raises a question about non-determinism and how it applies to the generated artifact. Non-determinism on TLA⁺ occurs on a state the *Next* formula is true for more than one step starting on that state - that is $\exists s1, s2 \in \text{states} \mid \text{Next}(s \rightarrow s1) \wedge \text{Next}(s \rightarrow s2)$. There are some different ways to go about it: the execution could be divided in branches running on different parallel processes, determined by a random factor, or prompt the user for a choice. This will depend heavily on the scenario in which the generated code is running.

The chosen approach to handle such non-determinism is to isolate it with a concept called oracle. Every generated code has an oracle process, and the main process sends a message to this oracle every time it needs to decide something. The oracle decision behavior can be implemented by the programmer and has a simple interface: it receives a message with a list of possible transitions to take, and responds with the chosen action.

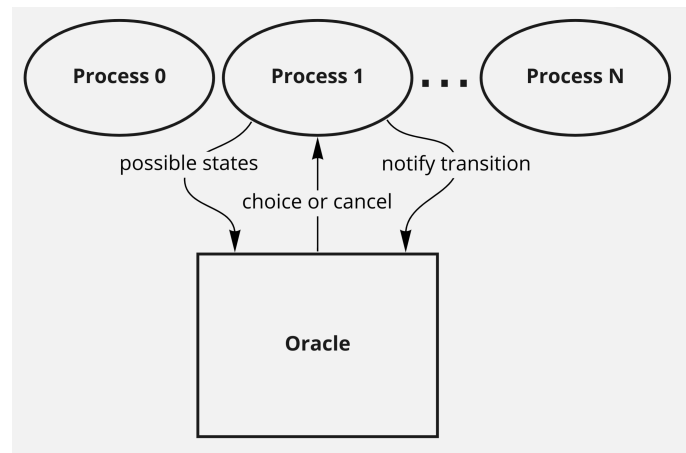


Figure 2: Interactions with the oracle

The set of possible transitions sent to the oracle has only transitions from enabled actions, that is, actions whose conditions are satisfied on the current state. A scenario where multiple actions are enabled originates from the use of the or ($\vee$) connective, and will always be translated to a `decide()` function call with a structure mapping the actions conditions to its transitions. The `decide()` function will filter this structure, selecting only the actions whose condition is true. If a single action is enabled, it is a deterministic scenario, and that action is chosen. If more than one action is enabled, it sends all enabled actions to the oracle and waits for a response. The final transition is then notified to the oracle in case

there is any internal state to be updated. This procedure is described by Algorithm **algorithm 1** and illustrated at Figure 2.

From a concurrency point of view, the oracle serializes messages so that the generated code is only processing one thing at a time. This is consistent with the definition that all non-determinism is isolated to the oracle, since concurrency is a kind of non-determinism. Deterministic segments of the spec - sequences of transitions where only one action is enabled at the time - can be viewed as critical zones or atomic operations, which are crucial to handle concurrent systems consistently.

Algorithm 1 Algorithm for the main function

```

1: procedure MAIN(variables)
2:   possible_actions  $\leftarrow$  next()            $\triangleright$  actions with satisfied
   conditions
3:   distinct_states  $\leftarrow$  unique states from transitions in
   possible_actions
4:   if size of possible_actions = 1 then            $\triangleright$  Deterministic
5:     notify oracle of transition to first state from
       distinct_states
6:     return main(first state from distinct_states)
7:   else                                            $\triangleright$  Non-deterministic
8:     send possible_actions to the oracle
9:     chosen_action  $\leftarrow$  oracle response
10:    notify oracle of transition to state from chosen_action
11:    return main(state from chosen_action)

```

4 TEST GENERATION

Besides producing code, the tool is also able to produce tests for that code using the same TLA⁺ specification as input. While the code generator uses the model specification alone, a test generator can benefit from the usage of other information such as specified properties and interactions with a model checker. Using these additional information to obtain valid behaviors accordingly to the specification can then be used to test the generated code, since a valid behavior for the specification must be valid in the implementation. If any behavior allowed by the specification is not allowed in the implementation, there is an error on the translation process.

After obtaining test scenarios, the generator must be able to couple them with the code in order to run them. The scenario's initial state has to become an input, and the execution behavior of the program must be comparable with the sequence of states from the rest of the scenario. Finally, some coverage criteria must be used to measure the quality of the generated test suite in order to guide the development of the tool.

To exemplify some techniques, the didactic TLA⁺ specifications for multiple traffic semaphores is given in Figure 3. These specifications consider a set of semaphores under the following requirements:

- (1) The sequence of colors for each semaphore should be always “red”, “green”, “yellow” and back to “red”;
- (2) Two different semaphores should never be “green” at the same time;
- (3) Opening of semaphores (turning “green”) should alternate.

MODULE *TrafficSemaphores* ┌
 EXTENDS *Integers*, *FiniteSets*
 VARIABLE *colors*, *next_to_open*
 CONSTANT *SEMAPHORES*

$$\begin{aligned} \text{TurnGreen}(s) &\triangleq \wedge \forall s2 \in \text{SEMAPHORES} : \text{colors}[s2] = \text{"red"} \\ &\quad \wedge \text{colors}' = [\text{colors} \text{ EXCEPT } ![s] = \text{"green"}] \\ &\quad \wedge \text{next_to_open} = s \\ &\quad \wedge \text{next_to_open}' = (s + 1) \\ &\quad \% \text{Cardinality}(\text{SEMAPHORES}) \end{aligned}$$

$$\begin{aligned} \text{TurnYellow}(s) &\triangleq \wedge \text{colors}[s] = \text{"green"} \\ &\quad \wedge \text{colors}' = [\text{colors} \text{ EXCEPT } ![s] = \text{"yellow"}] \\ &\quad \wedge \text{UNCHANGED } \langle \text{next_to_open} \rangle \end{aligned}$$

$$\begin{aligned} \text{TurnRed}(s) &\triangleq \wedge \text{colors}[s] = \text{"yellow"} \\ &\quad \wedge \text{colors}' = [\text{colors} \text{ EXCEPT } ![s] = \text{"red"}] \\ &\quad \wedge \text{UNCHANGED } \langle \text{next_to_open} \rangle \end{aligned}$$

$$\begin{aligned} \text{Init} &\triangleq \wedge \text{colors} = [s \in \text{SEMAPHORES} \mapsto \text{"red"}] \\ &\quad \wedge \text{next_to_open} = 0 \end{aligned}$$

$$\begin{aligned} \text{Next} &\triangleq \exists s \in \text{SEMAPHORES} : \\ &\quad \text{TurnGreen}(s) \vee \text{TurnYellow}(s) \vee \text{TurnRed}(s) \end{aligned}$$

$$\text{Spec} \triangleq \text{Init} \wedge \Box [\text{Next}]_{\langle \text{colors}, \text{next_to_open} \rangle}$$

Figure 3: TLA⁺ Specification for multiple traffic semaphores working together

In this spec, alternation is controlled by the *next_topen* variable, which cycles through the list of semaphores, and a specific semaphore can only turn green if all semaphores are red, and it is the next to open. The following properties are defined for the specification based on the requirements:

$$no_collision \stackrel{\Delta}{=} \text{Cardinality}(\{s \in SEMAPHORES : colors[s] = \text{“green”}\}) \leq 1$$
$$never_stuck \stackrel{\Delta}{=} \text{WF}_{\langle colors \rangle}(Next) \Rightarrow \\ \forall s \in SEMAPHORES : \Diamond(colors[s] = \text{“green”})$$

no_collision is an invariant establishing that at every state, the number of green semaphores should be less than or equal to 1. *never_stuck* is a temporal formula asserting that if weak fairness of the *Next* state action for the variable *colors* holds, then all semaphores should eventually be green. The weak fairness condition for this property is necessary since it would never hold when stuttering steps over *colors* occur forever.

4.1 Specification information to obtain scenarios

The specified model contains enough information about the transition system to generate tests for every state or every transition. However, using only the model information can be inefficient since the problem of finding routes minimizing revisited states is a NP-Hard problem, as shown in [6]. Additionally, obtaining scenarios directly from the TLA⁺ spec would be susceptible to errors on the

translation process between TLA⁺ structures and test structures. The transition system could rather be obtained directly from the TLC model checker, which exhaustively explores all states and is able to output which states were visited.

Besides the model, a specification should contain properties to be verified. Section 7 mentions work related to model-based and property-based testing. A property can be translated directly to tests, where an invariant can be an assertion on each state of any execution, and temporal properties can check states accordingly to the timeline it imposes. Properties can also be used to generate tests in conjunction with the model checker by the obtainment of counterexamples. The counterexample could be obtained either by the TLC or the Apalache model checker.

Both model checkers also provide simulation functionality, - which can be used to obtain samples of traces that respect the model, without a property dependency. The simulation functionality is a viable alternative for witness-based test generation when there are few or no good properties specified.

Some information required for generating tests is obtained from model checker runs, and this work depends on information that is provided exclusively by the model checker TLC. TLC is an explicit state model checker, and therefore enumerates all states and transitions while model checking a specification. The produced information about states and transitions is the main source of information for generating unit tests, and this information is not available in the Apalache model checker as it does not enumerate all states. For counterexample information, both model checkers can be used to obtain it, as the output format for them is similar. Therefore, one can benefit from trying both tools and use the most performant one for a spec to generate counterexample-based tests from.

4.1.1 States explored by TLC. When running TLC, it is possible to ask it to dump explored states to a file. It can either provide a list of states with variables values or compose the states in a graph such as the one in Figure 4 generated from the Traffic Semaphores spec with *SEMAPHORES* = 0, 1. Any tour on this graph beginning on a valid initial state should be possible in the implementation - if not, then the implementation is incorrect.

A limitation of this graph generation feature is that it is not able to inform which actions are used to generate each transition. The only information about the transition is the two states it is connecting. Therefore, in order to apply these transitions to the implementation, there are two options:

- (1) Derive which action would make the transition possible from the TLA⁺ model and choose it, or
- (2) Inspect the internal state that would result from each possible action and choose the one that results in the transition result state.

Option 2 was shown to be a good interface solution as it was also useful for handling counterexample information. Figure 5 a unit test example with this strategy.

4.1.2 Counterexample found by the model checker. When asking a model checker to check properties that are not satisfied, it will try to produce a counterexample for that property. This counterexample is a behavior (sequence of states) that is allowed by the model and does not satisfy the property.

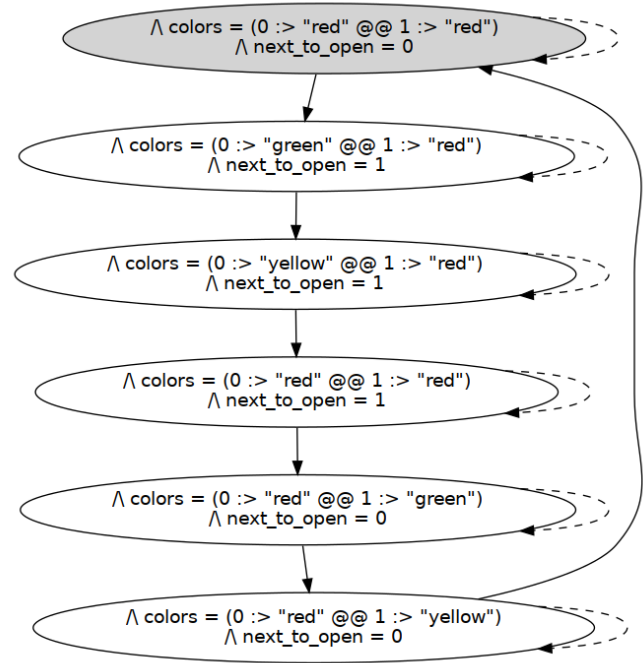


Figure 4: State graph for the traffic semaphore spec generated by TLC. Gray states are initial states.

```
test "fromState 7" do
  variables = %{
    colors: %{0 => "red", 1 => "red"},
    next_to_open: 0
  }

  expectedStates = [
    %{
      colors: %{0 => "green", 1 => "red"},
      next_to_open: 1
    }
  ]

  actions = List.flatten([TrafficSemaphores_main.next(variables)])
  states = Enum.map(actions, fn action -> action[:state] end)

  assert Enum.sort(Enum.uniq(states)) == Enum.sort(Enum.uniq(expectedStates))
end
```

Figure 5: A single unit test generated from a state graph's node

A usual specification, however, will not have any unsatisfied properties ready to be used. To obtain unsatisfied properties that can be used to generate counterexamples, the provided satisfied properties have to be negated and turned into trap properties. A property can also be decomposed into smaller properties to generate different (and often faster) counterexamples. One simple way of altering the property to obtain different witnesses is to negate different parts of it.

Taking the *never_stuck* property for example, it is an implication $A \Rightarrow B$ and thus can be rewritten as $\neg A \vee B$. Negating the whole property would result in $A \wedge \neg B$, so any state that satisfies B or does not satisfy A would be part of a counterexample - and this

is too general, often the initial state will be a valid counterexample for itself, which is not an interesting behavior to check. Generally, a counterexample will be better if it is a longer sequence, since it would be testing more transitions in the implementation. In this case, another possible negation would be $A \Rightarrow \neg B$. A witness for this trap property would be a sequence where A holds but B does not, which is more restrict and therefore will take more transitions to be falsified and thus can generate a better counterexample. Therefore, the trap property for *never_stuck* is build with the negation of the implication result, ensuring that the witness occurs in a behavior where the weak fairness holds:

$$\text{trap} \triangleq \text{WF}_{\langle \text{colors} \rangle}(\text{Next}) \Rightarrow \\ \neg \forall s \in \text{SEMAPHORES} : \Diamond(\text{colors}[s] = \text{"green"})$$

Checking this trap property with TLC yields the following sequence as counterexample, informing the action name that led to each state:

```
State 1: <Initial predicate>
/\ colors = (0 :> "red" @@ 1 :> "red")
/\ next_to_open = 0

State 2: <TurnGreen>
/\ colors = (0 :> "green" @@ 1 :> "red")
/\ next_to_open = 1

State 3: <TurnYellow>
/\ colors = (0 :> "yellow" @@ 1 :> "red")
/\ next_to_open = 1

State 4: <TurnRed>
/\ colors = (0 :> "red" @@ 1 :> "red")
/\ next_to_open = 1

State 5: <TurnGreen>
/\ colors = (0 :> "red" @@ 1 :> "green")
/\ next_to_open = 0

State 6: <TurnYellow>
/\ colors = (0 :> "red" @@ 1 :> "yellow")
/\ next_to_open = 0

State 7: <TurnRed>
/\ colors = (0 :> "red" @@ 1 :> "red")
/\ next_to_open = 0

State 8: <TurnGreen>
/\ colors = (0 :> "green" @@ 1 :> "red")
/\ next_to_open = 1

State 9: <TurnYellow>
/\ colors = (0 :> "yellow" @@ 1 :> "red")
/\ next_to_open = 1

State 10: <TurnRed>
/\ colors = (0 :> "red" @@ 1 :> "red")
/\ next_to_open = 1
```

Back to state 5: <TurnGreen>

If a sequence of actions like this is produced as a valid counterexample for a model, then that model's implementation should allow this exact sequence of actions, and therefore an automated test can be done by providing to the implementation the necessary inputs that will lead to this behavior and checking if the behavior can be executed.

On the strategy for deriving trap properties, it is not necessary to elaborate heuristics to improve property negation in order to obtain better counterexamples. The model checker can check different negations for the same property and the test generator tool can choose, between feasible properties, which counterexamples to use by a simple criterion such as sequence size. However, this is a step of test generation with low manual effort and big impact on the results, so it is left for the tool user to decide which properties and their counterexamples or simulation results are more relevant for testing, and automation of this selection can be explored in a future work.

4.2 Test Harness

From behaviors obtained either from the state exploration graph or a counterexample, executing tests mean to check that each behavior is executable by the implementation. Since this work focuses on generating tests for code that is also generated, the harness here proposed is restricted to these code interfaces, but it is possible to adapt the proposed strategies to other implementations with similar interfaces.

Generated tests are static, and only need to be re-generated if the model or the properties change. There is no coupling with the model checker or the test generator after the tests are generated. The test suite runs independently within the Elixir project.

4.2.1 Black-Box vs White-Box. White-box tests check internal states while black-box tests are restricted to checking output. Black-box testing is implemented in this context by checking only stages with non-determinism, which on generated code turns into interaction with external factors through the oracle. The oracle can be used to choose an action accordingly to the behavior being tested, and the test should fail if the choice is not possible. These black-box tests act like integration tests in software engineering terminology, and can also be used as regression tests. When a bug is found in an application being developed, a test is often written reproducing the bug scenario and expecting the correct outcome, so further updates after the bug fix cannot re-introduce it without breaking a test. Model checking specifications of software can also find bugs, and this sort of test generation provides an automated form of verifying the bug scenario's outcome in the implementation.

In contrast, white-box tests act like unit tests, where all states of the implementation, including internal ones, are checked against model information. An internal state is any state that is not directly influenced by the user or exposed to them. In some applications of software testing, internal states are not accessible and thus white-box testing is not possible. In this work's scenario, however, both model and implementation internal states are available, since the specification is a requirement to test generation and the implementation is generated as the next function that can be called repeatedly to obtain any state.

4.2.2 Tools. The state graph produced by TLC is outputted in .dot file format. The dot CLI tool is then used to produce a JSON format that is then processed with jq to filter out some noise information. The result is a .json file to be parsed by the test generation tool.

Regarding parsing of specifications, the tool implements a prototype of a TLA parser, which evidenced how complex it is to parse pure TLA. The main parsing procedure in the final version takes, instead, a .json version of the specification produced by the Apache parser (which uses SANY, the TLC parser, under the hood). This format has a better defined structure and is much easier to parse.

To run tests, Elixir provides a built-in testing library called ExUnit [23] that is used to execute behaviors and make assertions.

5 RUNNING AND TESTING DECENTRALIZED ALGORITHMS

Obtaining executable correspondence of declarative formulas is not trivial. Considering distributed systems the target environment, there are two concepts that need to be present in the execution context: non-determinism and decentralization. Non-determinism is addressed by the oracle, which establishes a clear interface around pieces of code where non-determinism can occur. Yet the oracle alone is not sufficient to make the generated code decentralized.

In a centralized distributed system, with one central node and multiple distributed parts, the generated code runs actions from the central part of the model while distributed nodes can be interfaces communicating through the oracle. For example, in an IOT (Internet of Things) scenario, a central controller can run the model while many sensors send messages to the oracle.

But where does the model run when the specified system is completely decentralized? There is no specific node that is aware of every action. If the specification defines a producer and a consumer, for example, the executable correspondent of it should be divided in two parts: one responsible for the producer's actions and other for the consumer's. This division, however, cannot be derived from the specification alone, as TLA⁺ only defines actions and not their actors. Therefore, the generation tool provides a configuration interface to allow its user to define groups of actions that correspond to different nodes in a decentralized system.

The oracle's role in a decentralized scenario is also more complex. It acts as an abstraction for inter process communication by storing some user-defined shared variables and provides reads and updates, controlled by locks, to nodes in order for them to execute their actions. Each node starts independently, since they can be run at any time and place, then tries to obtain the lock from the oracle and read the full current state by combining its private variables with the shared variables obtained from the oracle. It then either performs an action or stays at the same state, informing the oracle about the result state, so it can update the shared variables' values and release the lock. If a node's actions have non-determinism between them, the node can have its own private oracle to handle this non-determinism without having access to any shared information at all.

Testing this decentralized generated code is also more complex. For white-box tests, instead of matching transitions from a state to a call of next() from that state, the test actually matches the

transitions to a combination of next() calls from all the generated nodes. The result is one white-box test file per specification testing all the generated files. These tests also act as a way to ensure that user-defined distribution of actions per node is complete, since if any action is unassigned, its transitions will be missing and the test will fail. This testing process is illustrated in Figure 6.

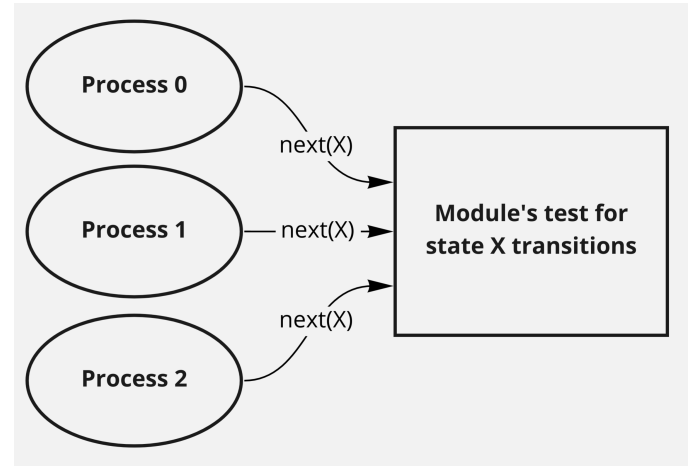


Figure 6: White-box tests for decentralized processes

Black-box tests run with an oracle implementation, so a single oracle is defined to work both as the node's private oracle and as the communication oracle. This oracle is more active than other ones which just wait for decision or lock requests, as it needs to start all nodes and control the order of execution in order to be able to follow the tested trace's path. It releases the lock to one node at a time, and waits for the resulting state: if it matches the next step on the trace, the transition is accepted and shared variables are updated; otherwise, it responds the node with a rejection, and queries the next one by releasing the lock for it. If the oracle is queried for a non-determinism resolution, it behaves the same by choosing only transactions that result in the desired state, when present. If, in a same step of the trace, none of the nodes can perform a transition that results in the next state of the trace, the test fails.

A black-box test representation is given at Figure 7. First, process 0 needs help from the oracle to solve non-determinism between X and Y, and the oracle looks at the trace it is checking and the current step counter, responding with X since it is the first state. Later, process 1 performs a transition to state Y and notifies the oracle of it. As the transition matches the trace's next state, the oracle accepts it and responds with OK. Then, process 2 notifies a transition to W, which is not expected and therefore refused with a "cancel" response - which does not fail the test until all other processes also have their actions canceled.

6 EXAMPLE: DIJKSTRA'S TERMINATION DETECTION

As a example for the code and test generation tool, Dijkstra's algorithm for termination detection in distributed computations, referenced as EWD840 [5], is explored. In this algorithm, nodes are

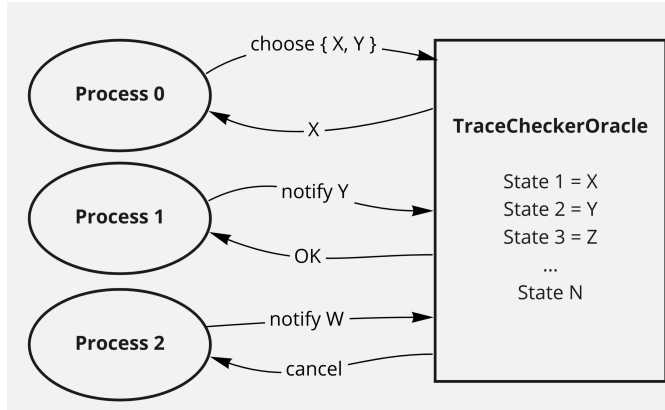


Figure 7: Example of interactions with the oracle in black-box testing

disposed in a ring, and the first node can initiate probes by sending messages to the last node with the goal of discovering whether there are active nodes in the ring. The message is a token which can be black or white. Each node has its own color, which can also be black or white. A node can be active or passive, and only active nodes can wake up other passive nodes - therefore, once all nodes are passive, they will stay passive forever, and this is the termination that should be detected by a probe initiated by the first node.

This is a decentralized system, so the generator tool should be configured to generate different files for each node in the system. In the TLA⁺ specification, actions for different nodes are parameterized by the node's index, and the distribution configuration can use this parametrization: Node 0 is responsible for actions referring to initiating the probe, waking up other nodes or deactivating itself; Other nodes cannot initiate a probe, but can pass the token to the preceding node, besides waking up other nodes and self-deactivating.

6.1 Generated code

Generating code for the specification of this algorithm instantiated with 3 nodes results in the elixir modules:

- (1) A base module containing all the translated definitions, some shared auxiliary functions, but no `next()`. This is an internal module that should not be used directly.
- (2) Three modules, one per node, containing the `next()` function for a node. This function calls other functions defined in the base module and should result in all possible transitions on a given state considering the actions under the node's responsibility. That is, node 0 `next()` should list transitions for initiating probes while others should not.
- (3) One task to start and monitor the oracle - a random oracle is used for quickest simulation - which starts with the translated initial state.
- (4) Three tasks to start each one of the nodes, also with the translated initial state and with the oracle's PID (Process Identifier) that is found by name from the BEAM's global registry. These tasks are generated separately because, in a

realistic scenario, different nodes can be started at different times and locations.

The four different tasks (one for the oracle and one for each of three nodes) can be run from different machines as long as they are on the same network. The oracle's address (BEAM's `sname`) is provided as a command-line parameter to the nodes tasks so it can be found on the network. With the oracle in place being the random oracle, once all tasks are started, a random simulation take place with messages being exchanged between the oracle and the nodes, and transitions being computed.

6.2 Generated tests

```
test "fromState 107" do
  variables = %{
    tpos: 1,
    active: %{0 => false, 1 => false, 2 => false},
    tcolor: "white",
    color: %{0 => "black", 1 => "black", 2 => "white"}
  }

  expectedStates = [
    %{
      tpos: 0,
      active: %{0 => false, 1 => false, 2 => false},
      tcolor: "black",
      color: %{0 => "black", 1 => "white", 2 => "white"}
    }
  ]

  actions =
    List.flatten([
      APAWD840_node0.next(variables),
      APAWD840_node1.next(variables),
      APAWD840_node2.next(variables)
    ])

  states = Enum.map(actions, fn action -> action[:state] end)

  assert Enum.sort(Enum.uniq(states)) == Enum.sort(Enum.uniq(expectedStates))
end
```

Figure 8: A single unit test generated from a state graph's node for the EWD840 specification

The white-box testing strategy generates 110 unit tests for this specification with 3 nodes, accounting for all 110 different states and 467 transitions. As an experiment, node 1's code is updated to perform no transition at all, and that causes a total of 68 tests to fail with missing transitions. One example for a state with only one transition is presented at Figure 8. This guarantees full state coverage and full transition coverage for the model.

As for the black-box tests, the most interesting witness to be tested in a termination detection algorithm is probably one that detects a termination. The property for termination detected is the following

$$\begin{aligned}
 \text{terminationDetected} &\triangleq \\
 &\wedge \text{tpos} = 0 \wedge \text{tcolor} = \text{"white"} \\
 &\wedge \text{color}[0] = \text{"white"} \wedge \neg \text{active}[0] \\
 \text{TerminationDetection} &\triangleq \\
 \text{terminationDetected} &\Rightarrow \forall i \in \text{Nodes} : \neg \text{active}[i]
 \end{aligned}$$

Negating the whole property results in a violation in the initial state, since the left side of the implication (*terminationDetected*) is not satisfied by it, making the implication true and its negation, false. Therefore, it is more useful to negate only the right side of the implication, resulting in a formula that asserts that if termination is detected, there is at least one active node. Running the model checker with this formula will result in a 7-state counterexample where termination is detected, but there are no active nodes, which is a witness to termination that can be used to test the code.

The test task starts a `TraceCheckerOracle` with this witness as trace, as well as the three nodes. The oracle restricts the nodes' transitions and choices, always respecting the oracle's interface, in order to follow the desired trace. After all 7 states from the trace have been reached, the oracle returns success and all processes are killed.

7 RELATED WORK

Code generation from specifications is an explored area, with different generation tools for many informal or semi-formal specification languages and few for formal ones. This section presents some works in this area that, even though are a generalization of the tool here proposed, are similar in the aspect that they present a translation from specifications to executable code.

The work in [15] generates C code from the formal requirements specification method SCR (Software Cost Reduction), using a translation method called APTS (Abstract Program Transformation System) with rewrite rules. [7] proposes a generation of C++ code from RSML (Requirements State Machine Language) specifications, providing a one-to-one mapping between the formal semantics of RSML and the implementation in C++. In [3], Java code is generated from Event-B formal modeling language. Another work proposes a Java base application generation from FAMM (Federation Architecture Metamodel) to assist on early prototyping and testing [1].

Considering this work's focus on concurrent systems, in [9] the authors propose a generation of parallel code in C with OpenMP from Signal programs. Signal is a synchronous multi-clocked language to describe safety critical systems, providing an abstraction of signals happening over a discrete time series. This is a scenario where it is important that the generated code can be executed by a parallel platform such as OpenMP, similarly to this work's distributed platform achieved by the BEAM virtual machine.

The work on [16] proposes a state-machine based model to specify transactions along with a model checker and code generator. The code generator isolates non-determinism to what the authors call handlers, similarly to what is proposed here with the oracle.

As for test generation from specifications, which is an instance of model-based testing, there are several different works. The following works are those that relate more closely to the proposal on this work.

The work in [8] uses a symbolic model checker such as SMV (Symbolic Model Verifier) to obtain states, controlling the state explosion problem. Models are defined as EFSMs (Extended Finite State Machines), and properties are written on CTL (Computation Tree Logic), a branching time temporal logic, to use the model checker to obtain counterexamples to be used as test scenarios. A

different approach in [24] uses the Java Pathfinder, a model checker for Java using the JVM, to generate test inputs to complex data manipulation. With the intent of efficiently generating tests while keeping high coverage, they use symbolic execution as a form of abstraction, where a set of constraints is calculated over the inputs for each possible executable path. The constraints are then solved to generate a minimal test suite covering all those paths.

The applied work in [25] generates tests from C code and requirements in form of properties. The C program is abstracted using predicate abstraction, and any unfeasible paths found later by the model checker can help to refine this abstraction. The model checker receives the abstraction and the requirements as trap properties, and outputs a counterexample in form of execution path. The model checker is used in [21] to generate characterization sets used in black-box testing. Characterization sets are useful in black-box testing since each input sequence on the set produces a different output for each different initial state.

The reviewed works showed different approaches to the test generation process using model checking, where some of them focused on improving efficiency, others on coverage criteria and many had objectives that were a combination of both, showing how they relate. The test generation works used a similar set of strategies with variations mainly involving efficiency improvements and area or language of application. The work proposed here also uses some of the observed strategies, with its primary variation being the application to the TLA⁺ language.

7.1 Related works in TLA⁺

There are no published works generating tests from TLA⁺ specification at the time of the writing of this paper, as of the authors' best effort to find them. However, some works in progress were presented at the 2020 TLA⁺ Conference, such as a model-based test generator based on TLC called Kayfabe [6]. This approach uses the model itself that TLC explores to make its own exploration, checking that every step of the program is a refinement of the model's state. In order to explore the entire state space efficiently, the method would need to find a state tour that minimizes revisited states, which the author shows reduces to the NP-Hard traveling salesman problem. It is also shown that test harness can have some limitations on how to map TLA⁺ values to concrete ones and on how to consider some program side effects that are not present on the model.

Another work, from Informal Systems, the company behind the Apache symbolic model checker for TLA⁺, obtains tests from counterexamples produced by Apache itself [11]. The test generation technique is part of the verification-driven development process (VDD) developed at the company, which begins with an English specification, that is manually translated into a TLA⁺ specification and later to code. The model-based testing reduces the gap between specifications and implementations in this flow.

8 CONCLUSION

Although TLA⁺ specifications are purely declarative, they are meant to be used to describe algorithmic behavior, and that should be simulatable, executable and testable. The main challenge is to define clear interfaces for the high level abstractions that TLA⁺ enables.

This work takes deep study in understanding TLA^+ concepts and how they work in an executable context, and turns it into a tool that translates all of these concepts (conditions, transitions, non-determinism, decentralization, state graphs and counterexamples) into artifacts more familiar to engineers: functions, tasks, inter-process communication, unit and integration tests.

The result is a full development environment generated from a single specification, that has the potential to help with learning TLA^+ , by working as a playground for people writing their first models, and mainly with reducing the gap between specification and implementation, by providing simple interfaces to connect a specification's code with a real world system to validate integrations and reproduce bugs (and their fixes, as regression tests).

The fact that generated tests are testing generated code takes the test harness burden off. Again, clear interfaces make for easy testing, where test files are self-containing (no external data needs to be read), a single function is called and the results are matched, resulting in high readability and low indirection. This is an environmental advantage when compared to generic model-based testing.

Future development of this work should focus on black-box testing and overall orchestration, as these can be useful features in multiple contexts. Black-box testing infrastructure can be used to bootstrap the system up to a certain point and then open it up for user-guided simulation, and that would be a great debugging tool as this is not possible with any model checker as of this writing. There are also improvements to be made on the current implementation, including mainly user experience related features and user documentation which were not prioritized in the first validation-focused implementation.

ACKNOWLEDGMENTS

This study was financed in part by the Fundação de Amparo à Pesquisa e Inovação do Estado de Santa Catarina - Brasil (FAPESC).

REFERENCES

- [1] Mehmet Adak, Okan Topçu, and Halit Oguztüzün. 2010. Model-based code generation for HLA federates. *Software: Practice and Experience* 40, 2 (2010), 149–175. <https://doi.org/10.1002/spe.949> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.949>
- [2] Joe L. Armstrong. 2003. Making reliable distributed systems in the presence of software errors.
- [3] Néstor Cataño. 2015. Code Generation for Event-B. *International Journal on Software Tools for Technology Transfer* (01 2015).
- [4] Ruby Community. 2021. *About Ruby*. <https://www.ruby-lang.org/en/about/>
- [5] Edsger W. Dijkstra, W.H.J. Feijen, and A.J.M. van Gasteren. 1983. Derivation of a Termination Detection Algorithm for Distributed Computations. In *Information Processing Letters*, Vol. 14. 217–219.
- [6] Star Dorminey. 2020. *Kayfabe: Model-Based Program Testing with TLA^+ /TLC*. https://conf.tlapl.us/2020/11-Star_Dorminey-Kayfabe_Model_based_program_testing_with_TLC.pdf
- [7] Mats Heimdahl and David Keenan. 1999. Generating Code from Hierarchical State-Based Requirements. (12 1999).
- [8] HS Hong, I Lee, O Sokolsky, and H Ural. 2002. A temporal logic based theory of test coverage and generation. In *TOOLS AND ALGORITHMS FOR THE CONSTRUCTION AND ANALYSIS OF SYSTEMS, PROCEEDINGS (Lecture Notes in Computer Science, Vol. 2280)*, Katoen, JP and Stevens, P (Ed.), Lab Verimag, SPRINGER-VERLAG BERLIN, HEIDELBERGER PLATZ 3, D-14197 BERLIN, GERMANY, 327–341. 8th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2002), GRENOBLE, FRANCE, APR 08-12, 2002.
- [9] Kai Hu, Teng Zhang, and Zhibin Yang. 2013. Multi-threaded code generation from SIGNAL program to OpenMP. *Frontiers of Computer Science* 7 (10 2013), 617–626. <https://doi.org/10.1007/s11704-013-3906-4>
- [10] Igor Konnov, Jure Kukovec, and Thanh-Hai Tran. 2019. TLA^+ Model Checking Made Symbolic. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 123 (Oct. 2019), 30 pages. <https://doi.org/10.1145/3360549>
- [11] Andrey Kuprianov and Igor Konnov. 2020. *Model-based testing with TLA^+ and Apache*. http://conf.tlapl.us/2020/09-Kuprianov_and_Konnov-Model-based_testing_with_TLA+_and_Apache.pdf
- [12] Leslie Lamport. 1983. Specifying Concurrent Program Modules. *ACM Trans. Program. Lang. Syst.* 5, 2 (1983), 190–222.
- [13] Leslie Lamport. 1994. The Temporal Logic of Actions. *ACM Trans. Program. Lang. Syst.* 16, 3 (1994), 872–923.
- [14] Leslie Lamport. 2002. *Specifying Systems: The TLA^+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley. <https://www.microsoft.com/en-us/research/publication/specifying-systems-the-tla-language-and-tools-for-hardware-and-software-engineers/>
- [15] Elizabeth I. Leonard and Constance L. Heitmeyer. 2008. Automatic Program Generation from Formal Specifications using APTS. In *Automatic Program Development: A Tribute to Robert Paige*, Olivier Danvy, Harry Mairson, Fritz Henglein, and Alberto Pettorossi (Eds.). Springer Netherlands, Dordrecht, 93–113. https://doi.org/10.1007/978-1-4020-6585-9_10
- [16] Anakreon Mentis and Panagiotis Katsaros. 2012. Model checking and code generation for transaction processing software. *Concurrency and Computation: Practice and Experience* 24, 7 (2012), 711–722. <https://doi.org/10.1002/cpe.1876> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.1876>
- [17] Stephan Merz. 2003. On the logic of TLA^+ . *Computers and Artificial Intelligence* 22 (01 2003), 351–379.
- [18] Robin Milner, Joachim Parrow, and David Walker. 1992. A calculus of mobile processes. I. *Information and Computation* 100, 1 (1992), 1 – 40. [https://doi.org/10.1016/0890-5401\(92\)90008-4](https://doi.org/10.1016/0890-5401(92)90008-4)
- [19] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. 2015. How Amazon Web Services Uses Formal Methods. *Commun. ACM* 58, 4 (March 2015), 66–73. <https://doi.org/10.1145/2699417>
- [20] C. A. Petri. 1962. Fundamentals of a Theory of Asynchronous Information Flow. In *IFIP Congress*. 386–390.
- [21] Christopher Robinson-Mallett, Peter Liggesmeyer, Tilo Muecke, and Ursula Goltz. 2006. Extended state identification and verification using a model checker. *INFORMATION AND SOFTWARE TECHNOLOGY* 48, 10 (OCT 2006), 981–992. <https://doi.org/10.1016/j.insof.2006.03.006> Workshop on Advances in Model-Based Software Testing, St.Louis, MO, MAY 15-16, 2005.
- [22] The Elixir Team. 2012. *Elixir*. <https://elixir-lang.org>
- [23] The Elixir Team. 2021. *ExUnit - Unit testing framework for Elixir*. https://hexdocs.pm/ex_unit/ExUnit.html
- [24] Willem Visser, Corina S. PĂSĂREANU, and Sarfraz Khurshid. 2004. Test Input Generation with Java PathFinder. In *Proceedings of the 2004 ACM SIGSOFT International Symposium on Software Testing and Analysis* (Boston, Massachusetts, USA) (ISSTA '04). Association for Computing Machinery, New York, NY, USA, 97–107. <https://doi.org/10.1145/1007512.1007526>
- [25] Songtao Xia, Ben Di Vito, and César Muñoz. 2005. Automated Test Generation for Engineering Applications. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering* (Long Beach, CA, USA) (ASE '05). Association for Computing Machinery, New York, NY, USA, 283–286. <https://doi.org/10.1145/1101908.1101951>
- [26] Brandon Zhang. 2017. PGo : Corresponding a high-level formal specification with its implementation.